SAP ABAP – Unit 5

Exercise 14 — Use a Structured Data Object

Purpose: Declare a structured attribute, fill it with a CDS SELECT statement, and access individual structure components in the output.

1. What You Learn

- How to define a local structure type with TYPES.
- How to declare an instance attribute using that structure type.
- How to access structure components using the hyphen (-) selector.
- How to use a structure as the target of a SELECT statement.
- How INTO @details differs from INTO (@field1, @field2, ...).
- Why a structure is useful when several related values belong together.

2. Exercise Overview

Task	What you do
Task 1	Copy the previous CDS exercise/template.
Task 2	Create structure type ST_DETAILS and attribute DETAILS.
Task 3	Read the structure components in GET_OUTPUT.
Task 4	Fill DETAILS directly from the CDS SELECT.
Final	Activate and run; verify all values in the console.

3. Starting Point

The previous exercise used three separate attributes for the database/CDS result:

airport_from_id TYPE /dmo/airport_from_id

airport_to_id TYPE /dmo/airport_to_id

carrier_name TYPE /dmo/carrier_name

Exercise 14 replaces these three scalar attributes with one structured object named DETAILS.

4. Task 1 — Copy Template

Use /LRN/CL_S4D400_DBS_CDS as the template, or continue from your working Exercise 13 class.

Suggested class name: ZCL_##_STRUCTURE

In your case, continuing in the existing ZCL_9309_LOCAL_CLASS is also possible if the training setup expects you to continue in the same class.

5. Task 2 — Declare a Structured Data Object

Inside the LOCAL TYPES tab, go to the PRIVATE SECTION of LCL_CONNECTION.

Create the local structure type ST_DETAILS:

```
PRIVATE SECTION.

  TYPES:
    BEGIN OF st_details,
      DepartureAirport  TYPE /dmo/airport_from_id,
      DestinationAirport TYPE /dmo/airport_to_id,
      AirlineName       TYPE /dmo/carrier_name,
    END OF st_details.

  DATA carrier_id    TYPE /dmo/carrier_id.
  DATA connection_id TYPE /dmo/connection_id.

  DATA details TYPE st_details.

ENDCLASS.
```

The old attributes are no longer needed:

```
* DATA airport_from_id TYPE /dmo/airport_from_id.

* DATA airport_to_id TYPE /dmo/airport_to_id.

* DATA carrier_name TYPE /dmo/carrier_name.
```

5.1 Structure Meaning

ST_DETAILS is a type definition. It is a blueprint, not the actual data. DETAILS is the actual instance attribute that stores the values.

Think of it as: ST_DETAILS = design of a container; DETAILS = the actual container.

6. Task 3 — Access Structure Components

Change GET_OUTPUT so it reads values from DETAILS. The hyphen connects the structure name to one of its components.

```
METHOD get_output.

  APPEND |------------------------------| TO r_output.
  APPEND |Carrier: { carrier_id } { details-AirlineName }| TO r_output.
  APPEND |Connection: { connection_id }| TO r_output.
  APPEND |Departure: { details-DepartureAirport }| TO r_output.
  APPEND |Destination: { details-DestinationAirport }| TO r_output.

ENDMETHOD.
```

Important: In Eclipse, after typing details- press Ctrl + Space and choose the component. Do not rely on manually typing the component name.

6.1 How the Component Selector Works

- details-AirlineName → airline name stored inside DETAILS
- details-DepartureAirport → departure airport stored inside DETAILS
- details-DestinationAirport → destination airport stored inside DETAILS

7. Task 4 — Fill the Structure with SELECT

The previous SELECT placed the three results into three separate target variables. Now the whole result is placed into DETAILS.

Keep the old SELECT commented if the exercise asks you to preserve it.

```
SELECT SINGLE
  FROM /DMO/I_Connection
  FIELDS DepartureAirport,
         DestinationAirport,
         \_Airline-Name
  WHERE AirlineID = @i_carrier_id
    AND ConnectionID = @i_connection_id
  INTO @details.

IF sy-subrc <> 0.
  RAISE EXCEPTION TYPE cx_abap_invalid_value.
ENDIF.
```

7.1 What INTO @details Means

The SELECT returns three values. Instead of naming three separate target variables, ABAP writes the result into the components of DETAILS.

SELECT value	Structure component
DepartureAirport	details-DepartureAirport
DestinationAirport	details-DestinationAirport
_Airline-Name	details-AirlineName

8. Optional Variant — INTO CORRESPONDING FIELDS OF

The training material also shows a safer explicit mapping variant. In that version, the SELECT field names are matched to structure component names.

```
SELECT SINGLE
  FROM /DMO/I_Connection
  FIELDS DepartureAirport,
         DestinationAirport,
         \_Airline-Name AS AirlineName
  WHERE AirlineID = @i_carrier_id
    AND ConnectionID = @i_connection_id
  INTO CORRESPONDING FIELDS OF @details.
```

The AS AirlineName is important in this variant because the CDS path expression _Airline-Name otherwise does not have the same component name as DETAILS-AirlineName.

9. Complete Local Class — Working Version

```
CLASS lcl_connection DEFINITION.

  PUBLIC SECTION.

    CLASS-DATA conn_counter TYPE i READ-ONLY.

    METHODS constructor
```

```
      IMPORTING
        i_carrier_id    TYPE /dmo/carrier_id
        i_connection_id TYPE /dmo/connection_id
      RAISING
        cx_abap_invalid_value.

    METHODS get_output
      RETURNING
        VALUE(r_output) TYPE string_table.

  PROTECTED SECTION.

  PRIVATE SECTION.

    TYPES:
      BEGIN OF st_details,
        DepartureAirport   TYPE /dmo/airport_from_id,
        DestinationAirport TYPE /dmo/airport_to_id,
        AirlineName        TYPE /dmo/carrier_name,
      END OF st_details.

    DATA carrier_id    TYPE /dmo/carrier_id.
    DATA connection_id TYPE /dmo/connection_id.
    DATA details       TYPE st_details.

ENDCLASS.


CLASS lcl_connection IMPLEMENTATION.

  METHOD constructor.

    IF i_carrier_id IS INITIAL OR i_connection_id IS INITIAL.
      RAISE EXCEPTION TYPE cx_abap_invalid_value.
    ENDIF.

    SELECT SINGLE
      FROM /DMO/I_Connection
      FIELDS DepartureAirport,
             DestinationAirport,
             \_Airline-Name
      WHERE AirlineID = @i_carrier_id
        AND ConnectionID = @i_connection_id
      INTO @details.

    IF sy-subrc <> 0.
      RAISE EXCEPTION TYPE cx_abap_invalid_value.
    ENDIF.

    me->carrier_id = i_carrier_id.
    me->connection_id = i_connection_id.

    conn_counter = conn_counter + 1.

  ENDMETHOD.


  METHOD get_output.

    APPEND |-------------------------------| TO r_output.
```

```
    APPEND |Carrier: { carrier_id } { details-AirlineName }| TO r_output.
    APPEND |Connection: { connection_id }| TO r_output.
    APPEND |Departure: { details-DepartureAirport }| TO r_output.
    APPEND |Destination: { details-DestinationAirport }| TO r_output.

  ENDMETHOD.

ENDCLASS.
```

10. Your Successful Eclipse Result

The following screenshot shows the completed Exercise 14 running successfully. The console proves that the structured object contains the airline name, departure airport, and destination airport.

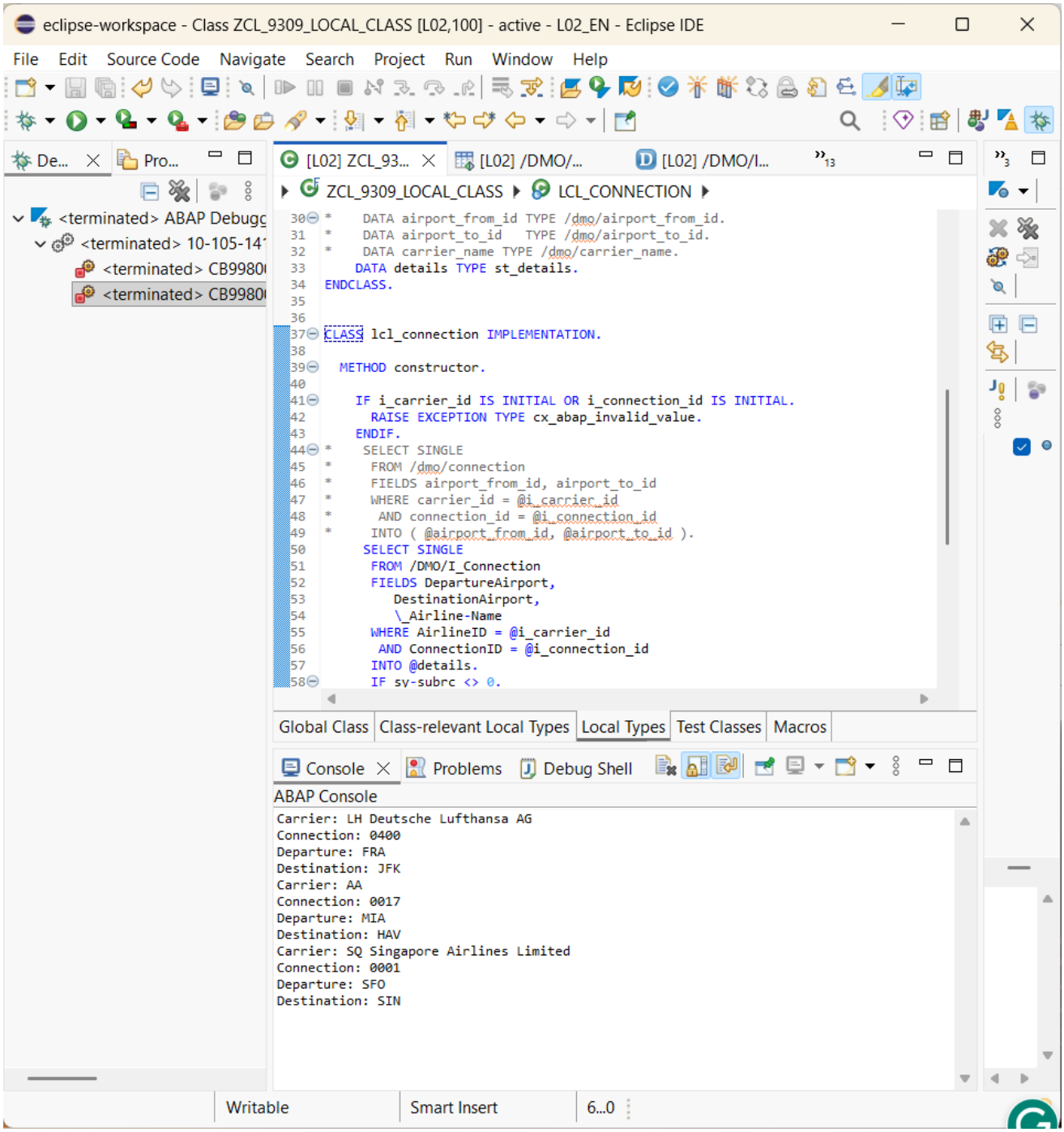

Figure 1 — Exercise 14 successful console output and code.

11. Expected Output

```
Carrier: LH Deutsche Lufthansa AG
Connection: 0400
Departure: FRA
Destination: JFK

Carrier: AA
Connection: 0017
Departure: MIA
```

```
Destination: HAV

Carrier: SQ Singapore Airlines Limited
Connection: 0001
Departure: SFO
Destination: SIN
```

12. Why This Exercise Matters

Without a structure, related values are spread across multiple variables. As the amount of related data grows, that becomes harder to maintain. A structure groups related fields under one meaningful object.

This pattern becomes especially useful when working with database records, CDS views, internal tables, method parameters, and larger business objects.

13. Common Mistakes

Mistake	Fix
Using the old attributes	If airport_from_id, airport_to_id, and carrier_name are commented out, they cannot be used in INTO.
Missing the period	SELECT must end with INTO @details. before the following IF statement.
Wrong component name	Use details-AirlineName, details-DepartureAirport, and details-DestinationAirport.
Forgetting @ before a host variable	Use @i_carrier_id, @i_connection_id, and @details in the SQL statement.
Confusing TYPE and DATA	ST_DETAILS defines a type; DETAILS is the actual data object.

14. Activation and Run Checklist

1. Check that ST_DETAILS is inside PRIVATE SECTION.
2. Check that DETAILS is declared as TYPE ST_DETAILS.
3. Check that old scalar attributes are removed or commented.
4. Check GET_OUTPUT uses DETAILS components.
5. Check SELECT ends with INTO @details.
6. Check IF sy-subrc <> 0 follows the SELECT.
7. Press Ctrl + F3 to activate.
8. Press F9 to execute.
9. Verify airline name, connection, departure, and destination in the console.

15. Key Concepts to Remember

ABAP syntax	Meaning
TYPES ... BEGIN OF ... END OF	Defines a structure type.
DATA details TYPE st_details	Creates an actual structure data object.
details-DepartureAirport	Accesses one component of the structure.
INTO @details	Places the SELECT result into the structure.
INTO CORRESPONDING FIELDS OF @details	Maps SELECT fields to structure components by matching names.
AS AirlineName	Gives the selected expression the component-compatible name needed for corresponding-field

	mapping.

Exercise 14 — Completion

Completed successfully: structure declared, components accessed, CDS data selected into the structure, class activated, and console output verified.